

Agentic Chatbots & MCP: Technical Q&A

This document summarizes the technical concepts regarding LangGraph Agents and the Model Context Protocol (MCP).

1. Why did the AI hallucinate clearing the chat history?

When asked to "forget everything", the AI responded that it had cleared the conversation history. However, it completely **hallucinated** this action. Because the ReAct agent is powered by a Large Language Model designed to be helpful, it tries to comply with your requests. However, it can **only** interact with your backend through the specific tools you provide it in the MCP server. Since we never gave it a `clear_chat_history` tool, it is impossible for it to actually delete records from the SQLite database—it just pretended it did! In production, AI engineers prevent this by adding a strict rule to the System Prompt telling the AI to instruct the user to click the "New Chat" button instead.

2. How did it calculate "next month this date" without a Python function?

The LLM inherently understands time and math! There is no Python regex or datetime function calculating "next month".

1. The API silently injects the current date into the system prompt behind the scenes.
2. The LLM's neural network parses the human phrase "next month this date" and does the calendar math internally.
3. It formats the resulting date perfectly as `YYYY-MM-DD` and passes it directly into the `create_subscription` MCP tool.

3. What is the difference between Demo 5 and Demo 6C?

Both use the exact same MCP tool registry, but the difference is **who controls the execution flow**:

- **Demo 5 (Workflow):** A human developer hardcodes the exact sequence of events (e.g., *Always call Tool A, then Tool B*). It is predictable but rigid.
- **Demo 6C (Agent):** The sequence is never written in advance. You give the LLM a goal (e.g., *"Save me money"*), and the LLM autonomously decides which tools to call, in what order, and dynamically evaluates the results (Reason + Act loop) until the goal is met.

4. Can an MCP server be hosted publicly?

Yes! MCP supports the **SSE (Server-Sent Events) over HTTP** transport layer. You can host your `mcp_server.py` script on a cloud provider like AWS or Render. Once it has a public URL (e.g., `https://my-tools.onrender.com/sse`), you can paste that link into your frontend Admin Panel, and your local LangGraph agent will connect to it over the internet.

5. Is MCP the same as ChatGPT Plugins?

Conceptually, **yes**. ChatGPT Plugins (or Custom GPT Actions) require you to write a custom OpenAPI JSON schema to tell ChatGPT how to hit your external REST API. **MCP is the open-source, universal version of that concept.** By building an MCP server instead of a proprietary ChatGPT plugin, your tools instantly become compatible with Claude Desktop, Cursor, LangGraph, and any other AI framework in the world.

6. How do I change this into a Job Application or Scheduler Agent?

The architecture is completely decoupled. To change the agent's entire purpose, you only need to change two things:

1. **The Persona:** Change `ASSISTANT_ROLE_DESCRIPTION` in `config.py` (e.g., *"You are an autonomous Job Assistant"*).
2. **The MCP Server:** Write a new `mcp_server.py` file with completely different tools (e.g., `@mcp.tool() def search_linkedin_jobs(keyword): ...` or `@mcp.tool() def book_google_calendar_meeting(date): ...`). You never need to touch the React UI or the core LangGraph loop!